

NEXUS

Knowledge Portal

An **AI-Powered** platform that turns repositories, documents, reusable code, and project tasks into an intelligent, **locally-hosted** knowledge base.

Discover knowledge. Understand projects. Build faster — with semantic search, Git intelligence, and context-aware AI chat.

STACK

React 19 · Vite · FastAPI
PostgreSQL + pgvector
Redis · RQ · Ollama
Mistral 7B · nomic-embed-text

MODE

Local-first AI
Docker Compose stack
Role-aware access

Project knowledge is **scattered** across too many places.

Modern software projects spread critical information across Git repositories, documents, PDFs, task boards, and tribal knowledge. Finding the right piece takes longer as projects grow — and the cost compounds.

SILOS

Docs live apart from code

Design docs, READMEs, tasks, code, and decisions each stay in their own tool. Nothing is interlinked or queryable together.

TIME

Manual search eats hours

Developers jump between tabs, files, and teammates to answer questions that should take seconds.

DISCOVERY

Reusable code stays hidden

Valuable functions and methods get rewritten over and over because nobody can find what already exists.

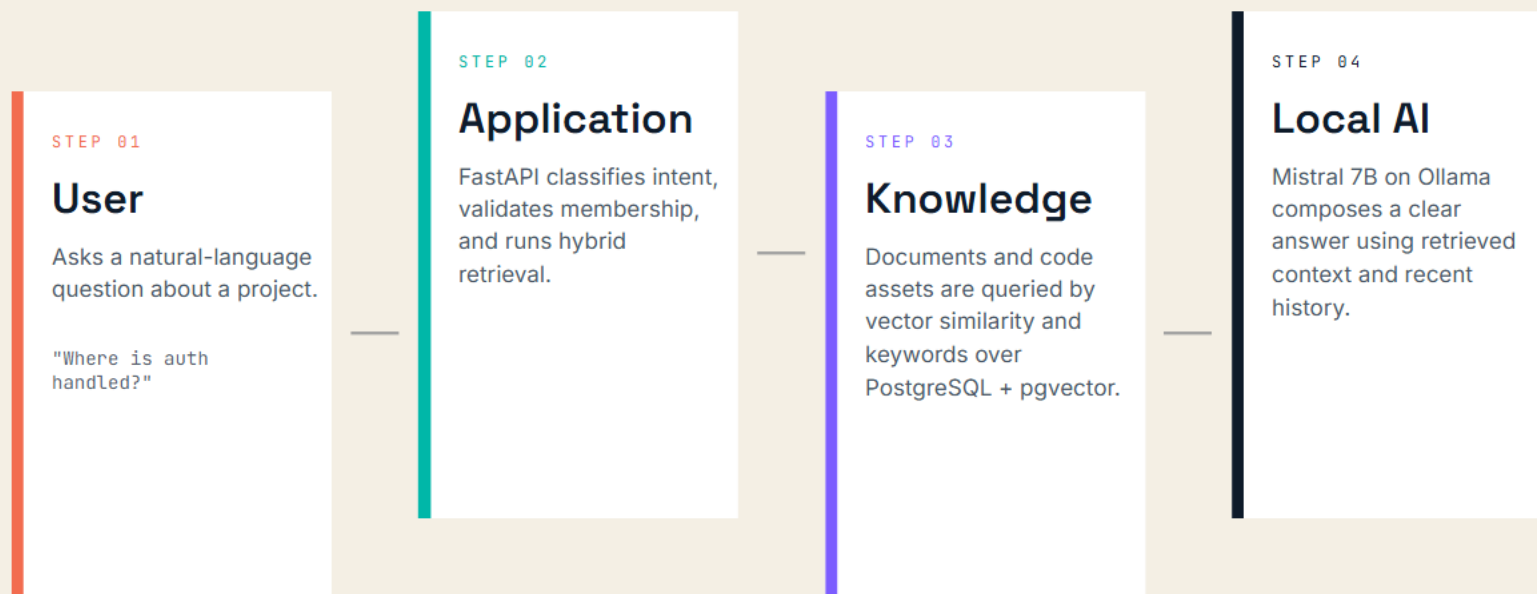
THE MANUAL APPROACH BECOMES THE BOTTLENECK — NEXUS TURNS PROJECT KNOWLEDGE INTO SOMETHING SEARCHABLE, RETRIEVABLE, AND AI-UNDERSTANDABLE.

One place to ask, to find, to act.

Nexus unifies everything people need to understand a project — and exposes it through natural-language questions answered by a local AI.

THE NEXUS LOOP

User → Application → AI → Knowledge → Answer



OUTCOME →

A response grounded in **actual project knowledge** — explainable, cite-able, and acted upon through tasks.

What Nexus is set out to do.

06

01 Unify scattered knowledge

Bring documents, repositories, code assets, and tasks into one discoverable layer.

02 Make answers retrievable

Replace keyword guessing with semantic + keyword hybrid search grounded in real project data.

03 Local & private AI

Run language and embedding models on the host through Ollama — no project data leaves the machine.

04 Surface reusable code

Detect functions and methods across repositories so good code gets reused instead of rewritten.

05 Conversational task creation

Turn natural-language asks into validated task drafts before they touch the project board.

06 Role-aware, observable, deployed

Honor project roles, run the entire stack in Docker Compose, and stay easy to inspect.

Six capabilities that make Nexus useful.



Document Knowledge

PDF · DOCX · MD · TXT · PNG · JPG

Upload files in any common format — the worker extracts text, detects structure, chunks content, and stores embeddings.



Git Repository Intelligence

CLONE · ANALYZE · EXTRACT

Clones repositories, detects languages, identifies entry points, reads the README, and extracts reusable code assets.



Hybrid Search

VECTOR + KEYWORD

Cosine similarity over chunks and code assets, ILIKE keyword match for tasks — combined into ranked results.



Context-Aware AI Chat

LOCAL · PERSISTENT · INTENT-DRIVEN

Persistent chat sessions classify intent, retrieve matching context, and answer with a local LLM that knows your project.



AI-Assisted Tasks

DRAFT · CONFIRM · PERSIST

Generate structured task drafts from a sentence, then confirm, modify, or cancel before anything is saved.



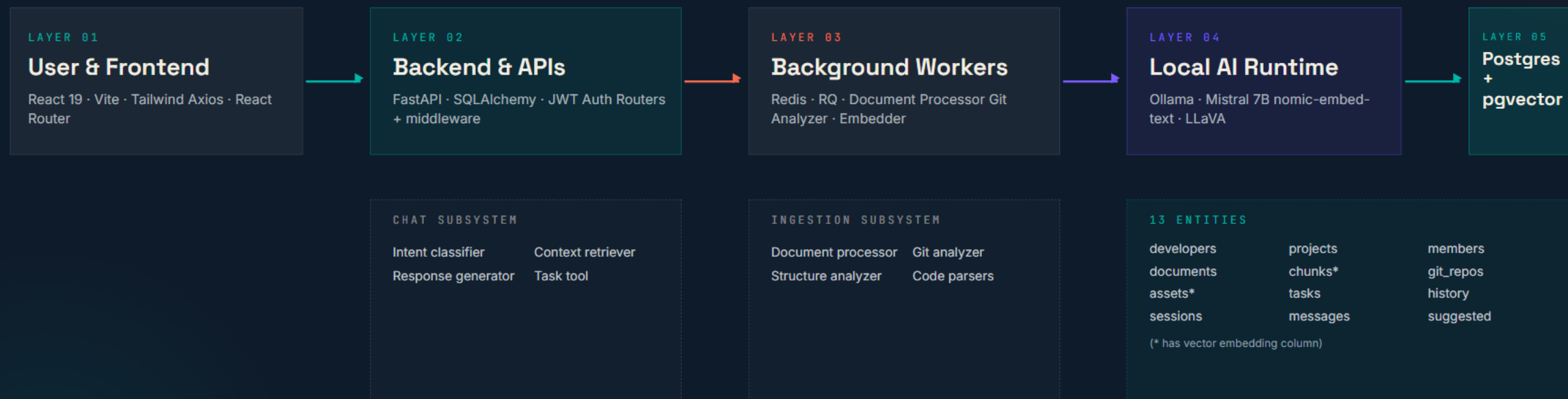
Role-Aware Access

MANAGER · ADMIN · TEAM LEAD

Privileged roles see more; everyone else is filtered by membership and the role attached to each asset.

A practical decomposition of **five** working layers.

User actions flow from React into FastAPI, fan out to background workers and an AI runtime, and converge on a single PostgreSQL store.



ALL FIVE SERVICES LIVE IN DOCKER COMPOSE · OLLAMA RUNS ON THE HOST

Compose

A pragmatic, modern stack — local-first, open, and boring on purpose.

FRONTEND

UI / UX

React 19

Vite

React Router

Tailwind CSS

Axios

Lucide React

@hello-pangea/dnd

React 19 SPA · Vite tooling · drag-and-drop task board · syntax-highlighted code display · markdown rendering

BACKEND

API / Services

FastAPI

SQLAlchemy

Pydantic

PyJWT

GitPython

pdfplumber

python-docx

Tesseract OCR

REST API + JWT middleware, async-capable, plus file parsers for PDF, DOCX, and OCR'd images

DATABASE

Storage

PostgreSQL 18

pgvector

Redis 7

RQ

One durable store for relational data, embeddings, and queue; cosine similarity search via pgvector

AI

Intelligence

Ollama

Mistral 7B

nomic-embed-text

LLaVA

All models pulled locally via Ollama — chat, embeddings, and (optional) vision. Nothing leaves the host.

INFRASTRUCTURE

DevOps

Docker

Docker Compose

uvicorn

Five services orchestrated with Compose — frontend, backend, worker, postgres, redis. Ollama runs as the host.

Two pipelines, one knowledge base.

Knowledge flows **in** through ingestion; answers flow **out** through retrieval and AI generation. Both meet in PostgreSQL.

PIPELINE 01 — INGESTION

From raw sources to embeddings



BRANCHING BY SOURCE

■ Document path

pdfplumber · python-docx · OCR via Tesseract →
800-token chunks with 100-token overlap

■ Repository path

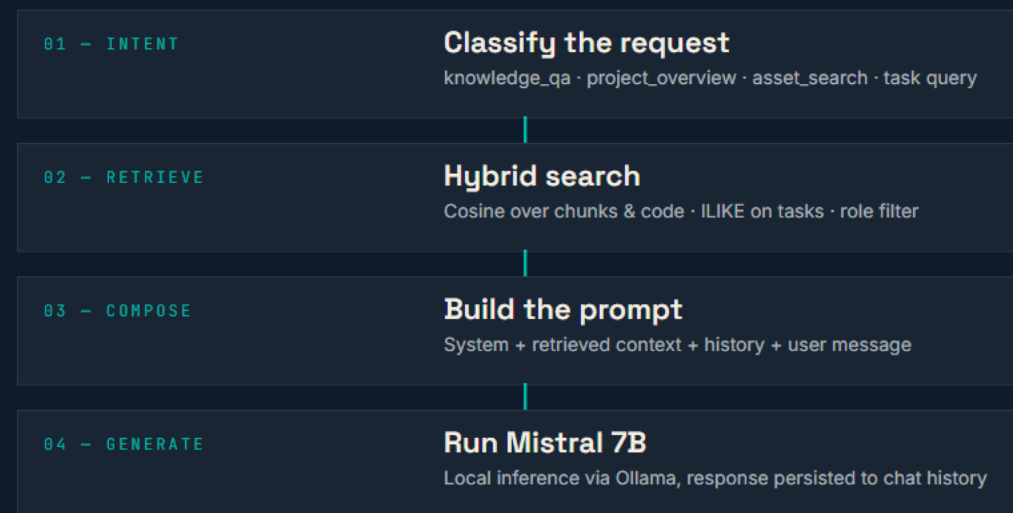
Clone → detect languages · extract README · parse
Python / Java · Kotlin · JS / TS

BOTH PATHS CONVERGE ON THE EMBEDDING STAGE

README sections and reusable code assets are
stored as `document_chunks` rows with
embeddings.

PIPELINE 02 — QUESTION ANSWERING

From question to grounded answer



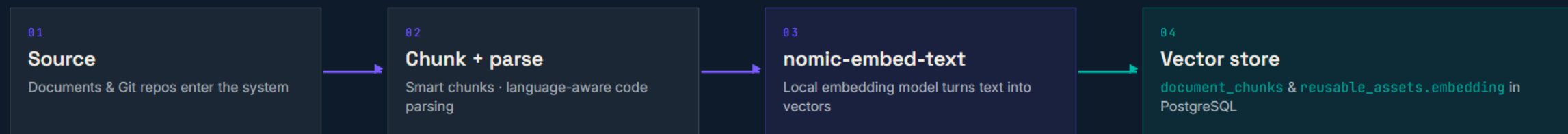
BOTH PIPELINES MEET IN STORAGE

A local RAG loop, tuned for code & docs.

Embeddings turn knowledge into vectors. Similarity search finds relevant pieces. The model composes a clear, grounded answer — entirely on your machine.

INDEXING

Build the index



ANSWERING

Answer the question



REUSABLE CODE ASSETS EMBED ON FULL_SIGNATURE + DOCSTRING · README SECTIONS LIVE AS DOCUMENT_CHUNKS

RAG

13 entities, one store.

PostgreSQL + pgvector — relational tables for structure, vector columns for similarity, projects at the centre.

DOMAIN GROUPS

01 • IDENTITY

developers

id • name • email • role • skills • projects • created_at

02 • WORKSPACE

projects & project_members

projects, project_members (creator flag)

03 • KNOWLEDGE

documents & git_repos

documents, document_chunks*, git_repos, reusable_assets*

04 • WORK

tasks & contributors

tasks, task_contributors, task_history

05 • AI CONVERSATIONS

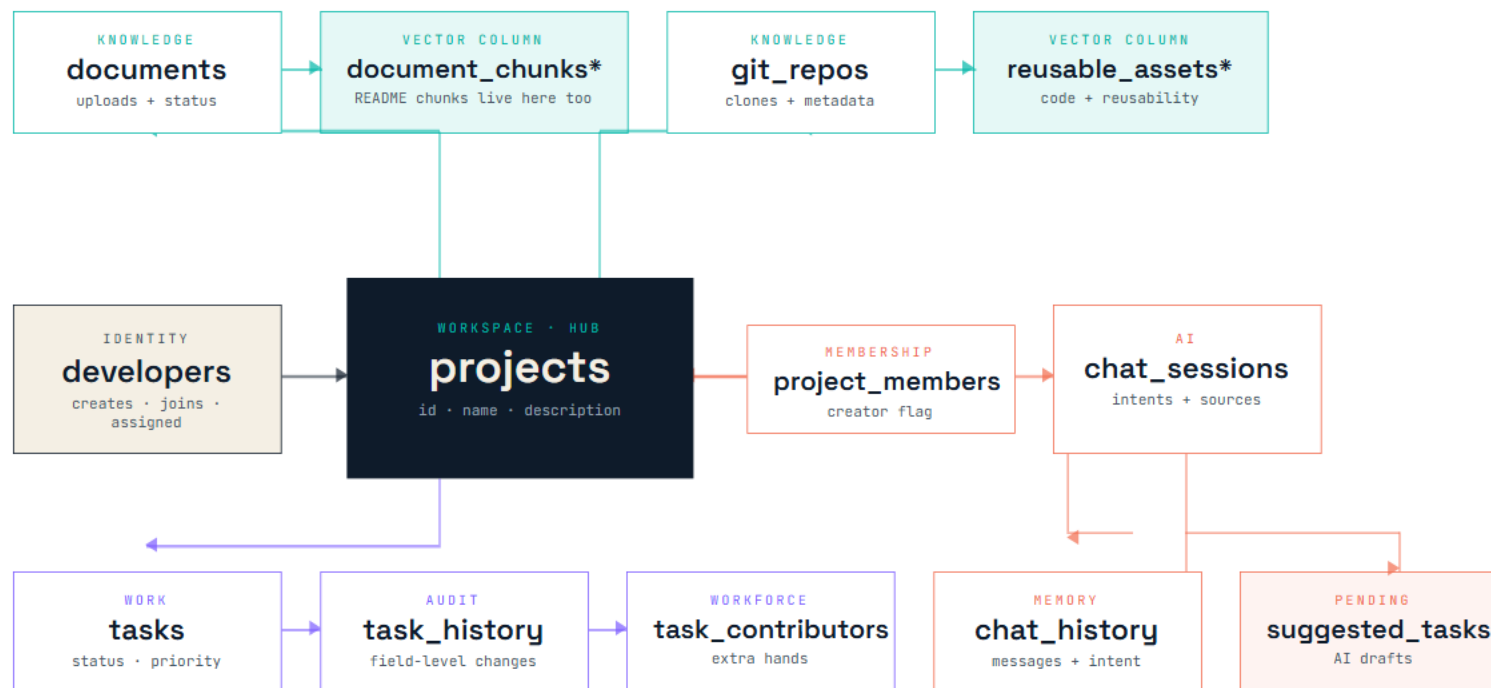
chat_sessions & drafts

chat_sessions, chat_history, suggested_tasks

RELATIONSHIP MAP

All 13 entities, projects at the centre.

— FK reference vector column



(*) ENTITY CARRIES A VECTOR COLUMN • ARROWS SHOW FOREIGN-KEY REFERENCES, NOT THE FULL REFERENTIAL GRAPH.

The codebase, broken into **five** working layers.

Each route, service, and integration maps to one of these. Together they cover the entire backend surface.

LAYER A

API Routes

FastAPI routers grouped by concern. Each is the public contract for one slice of the system.

API/AUTH.PY

Login & users

Authenticate a developer; list known users.

API/PROJECTS.PY · MEMBERS · DASHBOARD

Project CRUD

Create, list, update; manage members and dashboard.

API/DEVELOPERS.PY · TASKS.PY

People & work

Developer records and task lifecycle with status & priority validation.

LAYER B

Ingestion Services

- document_processor
- embedding_service
- readme_extractor
- git_analyzer coordinator
- structure_analyzer
- parsers (Python · Java · JS)

LAYER C

Search Services

hybrid_search merges three result sets — vector search across chunks and code assets, plus keyword ILIKE matching for tasks — sorted by relevance and trimmed by membership & role.

LAYER D

Chat Services

- intent_classifier
- response_generator
- context_retriever
- task_tool

LAYER E

Documents & Repos API

Upload, status polling, asset listing, asset detail & tree, plus deletion for documents and repositories — the bridge between user actions and the worker pipeline.

What a session looks like, end-to-end.

A Payment Gateway project team walks through onboarding, ingesting code, asking, and creating a task — all in one continuous flow.

01 Login & pick project
Authenticate, land on dashboard, select "Payment Gateway".

02 Ingest the repo
Provide a Git URL, the worker clones and analyzes it.

03 Upload docs
Drop PDFs / docs into the project to enrich the index.

04 Ask in plain English
"How does authentication work?" — a chat session starts.

05 Retrieve & generate
Hybrid search pulls context; Mistral answers on Ollama.

06 Create a task
AI drafts, user confirms, task persists + shows on the board.

SESSIONS STAY PERSISTENT • CHAT HISTORY IS QUERYABLE • ROLE CHECKS HAPPEN BEFORE EVERY READ

A REST surface, organised by what it touches.

Documented in OpenAPI · live at `/docs` · projectors under `/openapi.json`

SYSTEM

Health

GET `/health`

Health check for Docker & monitoring systems.

AUTH

Sessions & users

POST `/api/auth/login`

GET `/api/auth/users`

DEVELOPERS

Roster & records

POST `/api/developers`

GET `/api/developers`

GET `/api/developers/{dev_id}`

PROJECTS

Workspaces & membership

POST `/api/projects`

GET `/api/projects` · **GET** `/api/projects/personal-dashboard`

GET `/api/projects/{id}` · **PATCH** `/api/projects/{id}`

GET `/api/projects/{id}/members`

POST `/api/projects/{id}/members` · **DELETE** `/api/projects/{id}/members/{dev_id}`

DOCUMENTS & REPOSITORIES

Knowledge ingestion

POST `/api/projects/{id}/documents/upload`

POST `/api/projects/{id}/repos/upload`

GET `/api/projects/{id}/assets` · **GET** `.../assets/{asset_id}`

GET `.../assets/{asset_id}/content` · **GET** `.../tree`

DELETE `.../documents/{doc_id}` · **DELETE** `.../repos/{repo_id}`

TASKS

Work tracking

POST `/api/projects/{project_id}/tasks`

GET `/api/projects/{project_id}/tasks`

PATCH `/api/projects/{project_id}/tasks/{task_id}`

SEARCH

Hybrid project search

POST `/api/projects/{project_id}/search`

Filters: language, page, page_size

AI CHAT

Conversations

GET `.../chat/check-content`

POST `.../chat/session/start`

POST `.../chat/message`

GET `.../session/{session_id}/history`

Tokens, middleware, and role-aware filtering.

A Bearer token drives every request. Project membership and per-asset roles shape what each user actually sees.

AUTHENTICATION

JWT request lifecycle

01

Login → token issued

POST /api/auth/login returns a JWT, stored in localStorage.

02

Axios attaches the token

All API calls carry an Authorization Bearer header.

03

Middleware verifies

Decodes the token, places user_id & user_role into request state.

04

Routes enforce access

Reads user_role from state and applies membership & per-asset filtering.

AUTHORIZATION

Role-aware, applied at retrieval

PRIVILEGED ROLES

See more

manager

admin

team_lead

These roles can access broader project knowledge across the role spectrum.

OTHER ROLES

Filtered by asset

Each document and repository carries a role_access value. The search & retrieval pipeline hides content that doesn't match the requester's role.

RETRIEVAL PIPELINE

Five-step access sequence

user role → search request → membership check → role-based filter → vector search

Role checks happen before vector similarity so no one can fish knowledge out by guessing IDs.

Outcomes the platform already provides.

Drawn directly from implemented features — no aspirational claims, only what the system actually does today.

A

**Mixed-source knowledge,
one search bar.**

Documents, README sections, reusable code assets, and tasks all become queryable through a single hybrid endpoint.

B

**Local models that
answer in your stack.**

Mistral 7B runs on Ollama on the host. Project knowledge never leaves the host machine — handy for sensitive work.

C

**Reusable code surfaced
before it's rewritten.**

Detects Python, Java/Kotlin, and JS/TS functions, scores their reusability, and indexes them so good code wins.

D

**Conversations that
remember the project.**

Chat sessions persist with message history, intent, confidence scores, and source citations ready for the next turn.

E

**Tasks from chat,
guarded by a step.**

A natural ask becomes a structured draft. The user confirms, modifies, or cancels — no silent writes to the project.

F

**One stack to run,
all inside Docker.**

Five containers boot the system. Ollama is the only host-side dependency, configured by `OLLAMA_BASE_URL`.

In short — Nexus ingests project knowledge, makes it retrievable, and answers with a local AI grounded in your own code.

WHAT COMES NEXT

Realistic extensions, clearly labelled future work.

KNOWLEDGE INGESTION

Stay in sync

- Incremental Git sync
- Commit-aware indexing
- Better code chunking
- More language parsers

RETRIEVAL

Sharper answers

- BM25 + vector hybrid
- Reranking layer
- Query expansion
- Metadata filtering

AI

Smaller, capable

- Streaming responses
- Stronger citations
- Multi-model support
- Hallucination guardrails

TASKS + SECURITY

Plan & protect

- AI task prioritization
- Subtask generation
- Project-level auth
- Stronger JWT & CORS

HEADS UP Each item above is an enhancement, not yet shipped.

CONCLUSION

What we built, and why it matters.

01

A real problem, addressed concretely.

Project knowledge sat scattered across repos, docs, and task boards — Nexus unifies it.

02

A pragmatic stack, not a hype piece.

React · FastAPI · PostgreSQL · pgvector · Redis · Ollama — assembled to actually run.

03

Private by design.

Local models, role-aware retrieval, persistent sessions, containerized for easy isolation.

04

Extends cleanly.

The future scope above names the natural next steps, from reranking to subtask generation.

Discover knowledge.
Understand projects.
Build faster.